

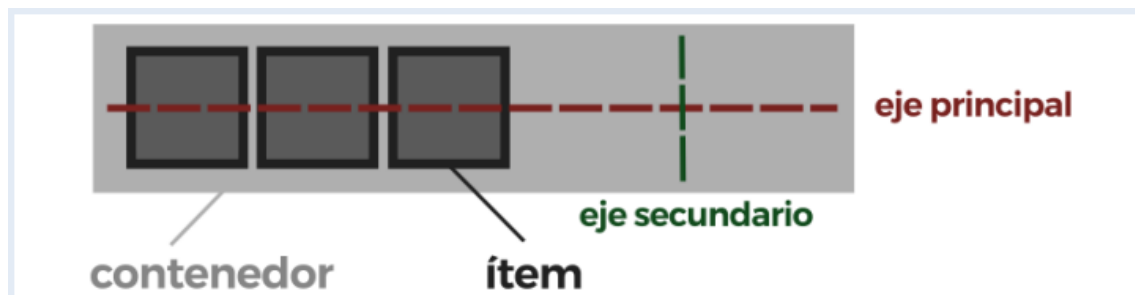
MODELOS DE DISEÑO

- **Flexbox:** Es un modelo de diseño unidimensional que sirve para distribuir el espacio en una interfaz y alinear elementos en filas o columnas.
- **Grid:** Es un modelo de diseño bidimensional mucho más potente, que permite controlar la estructura manejando filas y columnas a la vez.

CSS FLEXBOX

Conceptos Clave

- **Contenedor:** El elemento padre que agrupa los ítems.
- **Ítem:** Cada uno de los elementos hijos dentro del contenedor.
- **Eje principal (Main Axis):** Orientación principal.
- **Eje secundario (Cross Axis):** Perpendicular al principal.



Propiedades del Contenedor (Padre)

- **Activación: display: flex o inline-flex.** -> Tan pronto como hacemos esto, los hijos directos de este contenedor se vuelven ítems flex.

El resultado es que todos los ítems se alinean en una sola fila, usando el tamaño del contenedor como su tamaño en el eje principal. Si hay más ítems de los que caben en el contenedor, estos no pasarán más abajo sino que sobrepasarán el margen. Si hay ítems más altos que otros, todos los ítems serán ajustados en el eje cruzado, para igualar al ítem mayor.

- **flex-direction** (Define el eje principal):
 - **row**: Horizontal (por defecto).
 - **row-reverse**: Horizontal invertido.
 - **column**: Vertical.
 - **column-reverse**: Vertical invertido.
- **flex-wrap** (Controla el **comportamiento multilínea**): Aunque flexbox es un modelo unidimensional, es posible lograr que los ítems flex sean repartidos en varias líneas.

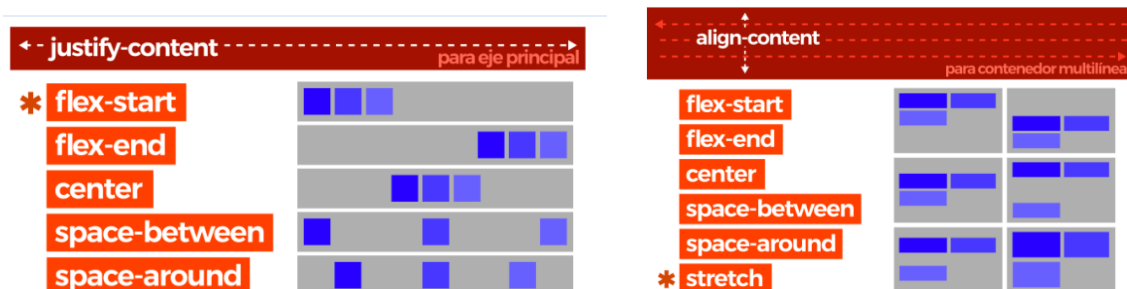
Haciendo esto, se deberá considerar cada línea como un nuevo contenedor flex. Cualquier distribución del espacio solo sucederá dentro de esa línea, sin referenciar las líneas colaterales.

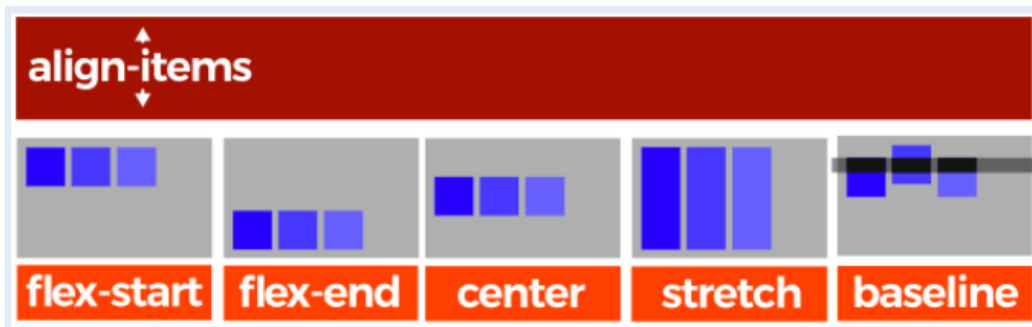
- **nowrap**: Una sola línea (los ítems se encogen si es necesario).
 - **wrap**: Multilínea (salto de línea si no caben).
 - **wrap-reverse**: Multilínea en dirección inversa.
- **flex-flow**: Atajo que combina flex-direction y flex-wrap (ej. row wrap).

```
.box{
  display: flex;
  flex-flow: row wrap;
}
```

- **Propiedades de alineación de ítems:**
 - **justify-content**: Alinea ítems en el *eje principal*. Valores:
 - **flex-start** -> Agrupa los ítems al principio.
 - **flex-end** -> Agrupa los ítems al final.
 - **center** -> Agrupa los ítems al centro.
 - **space-between** -> Distribuye los ítems dejando el mismo espacio entre ellos.

- **space-around** -> Distribuye los ítems dejando el mismo espacio a ambos lados de cada uno de ellos.
- **align-content**: Sirve para alinear cada una de las líneas del contenedor multilínea. Valores:
 - **flex-start** -> Agrupa los ítems al principio del eje principal.
 - **flex-end** -> Agrupa los ítems al final del eje principal.
 - **Center** -> Agrupa los ítems al centro del eje principal.
 - **stretch (por defecto)** -> Estira los ítems para ocupar de forma equitativa todo el espacio.
 - **Space-between** -> Distribuye los ítems desde el inicio hasta el final.
 - **Space-around** -> Distribuye los ítems dejando el mismo espacio a los lados de cada uno.
- **align-items**: Alinea ítems en el *eje secundario*. Valores:
 - **flex-start** -> Alinea los ítems al principio del eje secundario.
 - **flex-end** -> Alinea los ítems al final del eje secundario.
 - **Center** -> Alinea los ítems al centro del eje secundario.
 - **stretch (por defecto)** -> Alinea los ítems estirándolos de modo que cubran desde el inicio hasta el final del contenedor.
 - **Baseline** -> Alinea los ítems en el contenedor según la base del contenido de los ítems del contenedor.





- **Propiedades de los Ítems individuales (Hijos)**

- **align-self:** Sobrescribe la propiedad align-items del contenedor para un ítem específico (actúa exactamente igual).
- **flex-grow:** Factor que indica cuánto puede crecer un ítem respecto a los demás para ocupar el espacio disponible.
- **flex-shrink:** Factor que indica cuánto puede encogerse un ítem respecto al resto.
- **flex-basis:** Define el tamaño base del ítem antes de distribuir el espacio (auto, tamaño o content).
- **flex:** Atajo para las tres propiedades anteriores (flex-grow, flex-shrink, flex-basis).

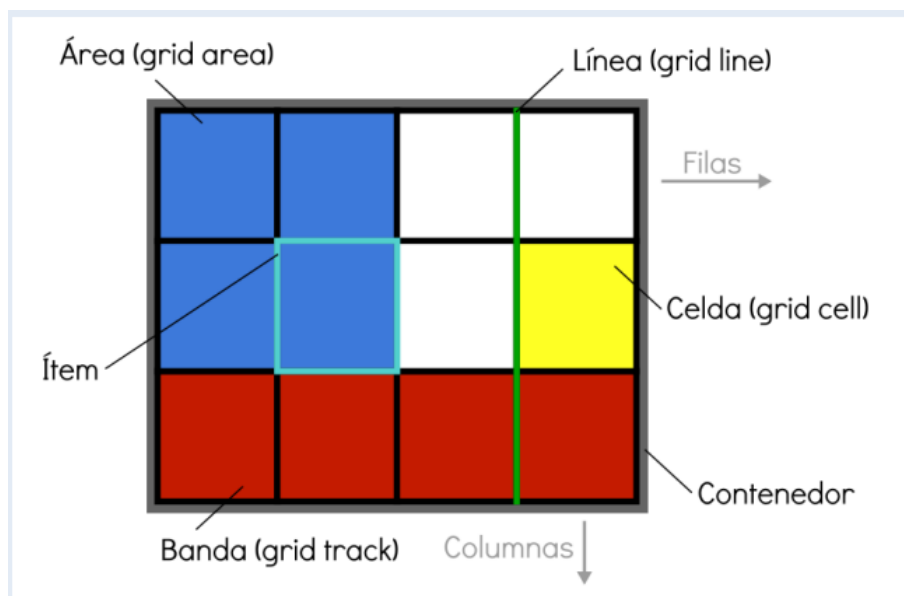
```
.item {  
    /* flex: <flex-grow> <flex-shrink> <flex-basis> */  
    flex: 1 3 35%;  
}
```

- **order:** Número que modifica el orden visual de los ítems. Si indicamos un order con un valor numérico, irá recolocando los ítems según su número. De esta forma podemos recolocar fácilmente los ítems incluso utilizando media queries o responsive design.

CSS GRID

Conceptos Clave

- **Contenedor:** Elemento padre (cuadrícula).
- **Ítem:** Cada uno de los hijos del contenedor. Es el elemento HTML real
- **Celda (grid cell):** La unidad mínima (cuadrado) de la cuadrícula.
- **Área (grid area):** Un conjunto o región de celdas.
- **Banda (grid track):** Una fila o columna completa de celdas.
- **Línea (grid line):** Los separadores horizontales o verticales entre celdas.



Propiedades del Contenedor (Padre)

- **Activación:** display: grid / display: block (comportamiento en bloque) o inline-grid / inline-block (comportamiento en línea).

Para utilizar cuadrículas Grid CSS, trabajaremos bajo este código HTML:

HTML

```
<div class="grid"> <!-- contenedor -->
  <div class="a">Item 1</div> <!-- cada uno de los ítems del grid -->
  <div class="b">Item 2</div>
  <div class="c">Item 3</div>
  <div class="d">Item 4</div>
</div>
```

- **Definir Filas y Columnas:**

- **grid-template-columns:** Define el tamaño de las columnas.
- **grid-template-rows:** Define el tamaño de las filas.
- **Unidades útiles:** px, %, auto, fr (fracción del espacio disponible).
- **Función repeat():** Permite repetir patrones. La expresión a utilizar sería la siguiente: repeat([número de veces], [valor o valores])

```
.grid {
  display: grid;

  grid-template-columns: 100px repeat(2, 50px) 200px;

  grid-template-rows: repeat(2, 50px 100px);
```

crearía una cuadrícula con 4 columnas (la primera de 100px de ancho, la segunda y tercera de 50px de ancho y la cuarta de 200px de ancho). Por otro lado, tendría 2 filas (la primera de 50px de alto, y la segunda de 100px de alto). En el caso de tener más ítems hijos, el patrón se seguiría repitiendo.

- **Grid por Áreas:** Mediante los grids CSS es posible indicar el nombre y posición concreta de cada área de una cuadrícula, permitiendo nombrar zonas del layout creando una "plantilla" visual con texto.
 - **grid-template-areas** -> Indica la disposición de las áreas en el grid. Cada texto entre comillas simboliza una fila.
 - **grid-area** -> Indica el nombre del área. Se usa sobre ítems hijos del grid.
 - Se usa la palabra none o puntos (.) para indicar una celda vacía.

```
CSS
C??
.grid {
  display: grid;
  grid-template-areas: "head head"
                      "menu main"
                      "foot foot";
}

a. { grid-area: head; background: blue }
b. { grid-area: menu; background: red }
c. { grid-area: main; background: green }
d. { grid-area: foot; background: orange }
```



- **Huecos (Gutters):** Por defecto, la cuadrícula tiene todas sus celdas pegadas a sus celdas contiguas. Aunque sería posible darle un margen a las celdas dentro del contenedor, existe una forma más apropiada, que evita los problemas clásicos de los modelos de caja: los huecos (gutters).

Para especificar los huecos (espacio entre celdas) podemos utilizar las propiedades `grid-column-gap` y/o `grid-row-gap`. En ellas indicaremos el tamaño de dichos huecos:

- **grid-column-gap:** Espacio entre columnas.
 - **grid-row-gap:** Espacio entre filas.
 - **grid-gap:** Atajo para definir ambos huecos simultáneamente.
- **Alineación en la Cuadrícula:**
 - **justify-items / align-items:** Distribuye los ítems individualmente en los ejes horizontal/vertical (start, end, center, stretch).

Propiedad	Valores	Descripción
<code>justify-items</code>	start end center stretch	Distribuye los elementos en el eje horizontal.
<code>align-items</code>	start end center stretch	Distribuye los elementos en el eje vertical.

Estas propiedades funcionan exactamente igual que sus análogas aplicadas al contenedor padre, sólo que sobre un elemento hijo.

- **justify-content / align-content:** También podemos utilizar estas propiedades sobre el contenedor padre para modificar la distribución de todo el contenido en su conjunto, y no sólo de los ítems por separado

Propiedad	Valores
<code>justify-content</code>	start end center stretch space-around space-between space-evenly
<code>align-content</code>	start end center stretch space-around space-between space-evenly

- **Grid Automático:** Es posible utilizar las propiedades `grid-auto-columns` y `grid-auto-rows` para darle un tamaño automático a las celdas de la cuadrícula. Para ello, sólo hay que especificar el tamaño deseado en cada una de las propiedades. Además, también podemos utilizar `grid-auto-flow` para indicar el flujo de elementos en la cuadrícula, y especificar por donde se irán añadiendo.
 - **grid-auto-columns / grid-auto-rows:** Define el tamaño automático para elementos que se añaden fuera del grid explícito.

- **grid-auto-flow:** Define el algoritmo de flujo de llenado (row, column, dense).

Propiedad	Valores	Descripción
grid-auto-columns	[tamaño]	Indica el tamaño automático de ancho que tendrán las columnas.
grid-auto-rows	[tamaño]	Indica el tamaño automático de alto que tendrán las filas.
grid-auto-flow	row column dense	Utiliza un algoritmo de autocolocación (intenta rellenar huecos).

- **Atajo: grid:** Por último, existe una propiedad grid que sirve de atajo para la mayoría de propiedades CSS relativas a cuadrículas. Su esquema de utilización sería el siguiente, junto a algunos ejemplos.

```

CSS
C??
.grid {
  /* grid: <grid-template> <grid-auto-flow> <grid-auto-rows> / <grid-
  auto-columns> */
  /* EJEMPLOS*/
  grid: 100px 20px;
  grid: 200px repeat(2, 100px) 300px;
  grid: row;
  grid: column dense;
  grid: row 200px;
  grid: row 400px / 150px;
}

```

Propiedades de los Ítems (Hijos)

- **grid-area:** Asigna un nombre al ítem para que ocupe el espacio definido en el contenedor mediante grid-template-areas.
- **justify-self / align-self:** Modifica la alineación horizontal/vertical únicamente de ese ítem específico (sobrescribe al contenedor).
- **Posicionamiento por Líneas:** Sirven para dictar exactamente dónde empieza y acaba un ítem basándose en los separadores.
 - **grid-column-start y grid-column-end** -> Indica en que columna empezará o acabará el ítem de la cuadrícula.

- **grid-row-start y grid-row-end** -> Indica en que fila empezará o terminará el ítem de la cuadrícula.
- **Atajos: grid-column y grid-row** (ej. grid-column: 1 / 3).
- **Palabra clave span**: Se usa para indicar que el ítem debe abarcar un número determinado de columnas o celdas (ej. span 3).

```
CSS
C??
.grid {
  display: grid;
}
.a {
  /* grid-column: <grid-column-start> <grid-column-end> */
  /* grid-row: <grid-row-start> <grid-row-end> */
  grid-column: auto;
  grid-column: 4 / 6;
  grid-column: span 3;
  grid-column: span 3 / 6;
}
```